School Van Tracking: Ordering Guarantees Under Unreliable Networks

Technical report, draft v0.3. | Anand Theertha | October 2026 | Repository: GitHub URL

Abstract

We study the telemetry protocol of a real-time school-van tracking app, where a driver's phone streams GPS positions to parents over unreliable mobile networks. Using TLA+ and the TLC model checker, we model a lossy, duplicating, reordering channel and check that the position shown to a parent never moves backward. TLC found a violation in the baseline protocol, caused by a delayed frame arriving after a newer one. We added sequence-number guards at the server and the parent client and re-checked the model: no violation was found within the explored bounds (3 frames, 24 distinct states). We also tested the implementation with injected duplicate, reordered, and batched frames. We describe the gaps between the model and the deployed system and the revisions needed to close them.

1. Problem

A parent who sees a stale or backward-jumping vehicle position may make a wrong decision about their child, so correctness matters more here than in most consumer apps. Vehicles pass through cellular dead zones, so connections drop, the driver app buffers positions, and frames are uploaded in bursts when the connection returns. In an early field test on a weak network, a dropped connection did not recover and a parent's map showed a stale, incorrect location. This report asks: what ordering guarantee can the system state, and how can it be checked before release?

2. System overview

The driver app samples GPS every 2 seconds and sends each position as a frame with a trip id, a sequence number (`seq_id`), a timestamp, and coordinates. The path is:

- Driver app (React Native) sends frames over Socket.IO on secure WebSockets.
- One or more Node.js ingestion nodes validate the token, persist the frame to PostgreSQL/PostGIS, update the current-trip cache, and publish it through a Redis pub/sub adapter.
- Parent apps subscribed to the trip receive the broadcast and render the position.

The update rate is bounded by the 2-second sampling interval (average end-to-end latency ~250ms on 4G). Appendix A gives the full stack and schema.

3. Baseline protocol and failure model

Baseline protocol. The server accepts each valid frame and broadcasts it in arrival order. Ordering relies on arrival time or the device timestamp. Device clocks drift and can be unsynchronized, so timestamps are not a safe basis for ordering.

Failure model.

- Connections can drop and reconnect.
- Frames can be duplicated by application-level retries.
- Frames from an old connection can arrive after frames from a new one.
- Several server nodes can process frames for the same trip concurrently.

Out of scope. GPS error, dead reckoning, map matching, and malicious clients.

4. Formal model (TLA+)

4.1 Channel abstraction and why it is justified

TCP and WebSockets do not reorder frames within one connection. The model nevertheless uses an unordered, lossy, duplicating channel, because that is a conservative abstraction of three real situations: old and new sockets overlapping during a reconnect, application-level retries, and parallel processing on multiple

server nodes. If a property holds under the weak channel, it holds under the real one.

4.2 State variables

- `driver_seq`: Sequence number of the latest frame the driver produced.
- `network_buffer`: Set of in-flight frames, each with a sequence number and payload.
- `server_ack_seq`: Highest sequence number the server has accepted.
- `parent_view_seq`: Sequence number of the position currently displayed to the parent.

4.3 Bounds

`MaxSeq` = 3 frames. Bounded to prevent infinite state space expansion.

5. Properties

- **Safety (action property):** the displayed position never moves backward.`MonotonicDisplay == [][parent_view_seq' >= parent_view_seq]_parent_view_seq`
- **Safety (invariant):** only positions the driver actually produced are displayed.`NoFutureFrame == parent_view_seq <= driver_seq`
- **Liveness (requires fairness):** the final position is eventually displayed.`FinalDelivery == (driver_seq = MaxSeq) ~> (parent_view_seq = MaxSeq)`

6. Results

6.1 Baseline protocol

TLC reports a violation of `MonotonicDisplay`. An abridged trace:

- **Driver sends frame 1**: `driver_seq` = 1, frame 1 in flight
- **Driver sends frame 2**: `driver_seq` = 2, frames 1 and 2 in flight
- **Server receives frame 2**: `server_ack_seq` = 2, parent shows 2
- **Server receives delayed frame 1**: Frame 1 is broadcast

- **Parent receives frame 1**: `parent_view_seq` goes from 2 to 1

Reordering alone produces this violation; duplication is not needed. Because `network_buffer` is an unordered set representing the asynchronous channel, frame 2 can be processed before frame 1, instantly violating the monotonic property.

6.2 Protocol with sequence-number guards

Configuration	MaxSeq	States generated	Distinct states	Depth	MonotonicDisplay
Baseline	3	78	34	6	VIOLATED
With guards	3	46	24	6	HOLDS WITHIN BOUNDS

A result of "holds" means no violation was found within these bounds. It is not a proof for all sizes.

6.3 Liveness

`FinalDelivery` **FAILS**. As expected, without an explicit acknowledgement and retransmission mechanism from the server to the driver, if the network drops the final frame, it is never resent. The parent view stalls indefinitely. This is a valid finding that highlights the trade-off of the v1 design: safety is guaranteed, but liveness is not.

7. Protocol revisions (v1 as built)

- **Server filter:** Drop any frame whose sequence number is less than or equal to the server's current highest seen value for that trip.
- **Parent guard:** The parent client renders a frame only if its sequence number is greater than the last one displayed.

8. Implementation test (fault injection)

A test harness uses a Socket.IO client to send scripted frame sequences to the server and records what a simulated parent displays.

Test	Injected fault	Expected after fix	Baseline Result	v1 Result
T1	Old frame arrives after a newer one	No backward jump	14 jumps	0 jumps
T2	Duplicate frame	No visible change	UI flickers	Ignored
T3	Burst of buffered frames after reconnect	Final position correct	Jumps	Correct
T4	Driver app restarts and counter resets	Frames accepted	Accepted	Dropped (Stale)
T5	Two server nodes receive frames concurrently	No backward jump	Jumps	Race Condition
T6	Last frame lost before reconnect	Final position shown	Stalls	Stalls

9. Limitations & Future Work (v2 Design)

The model and the v1 design leave several gaps that we plan to address in a v2 protocol:

- **Session Epoch:** If the driver uninstalls and reinstalls the app mid-trip, the local `seq_id` resets to 0. Without a session epoch, every subsequent frame looks stale and the trip silently freezes (as seen in Test T4).
- **Idempotent History:** Currently, dropping stale frames at the ingestion layer leaves gaps in the PostGIS historical log. In v2, every frame should be stored with a unique constraint on `(trip, epoch, sequence)` and inserted with "do nothing on conflict," separating historical integrity from the real-time broadcast.
- **Shared Server State:** `server_ack_seq` is currently tracked per node. With multiple stateless Socket.IO nodes, checking and updating this sequence

requires an atomic Redis Lua script to prevent race conditions (as seen in Test T5).
- **Acknowledgements and Retransmission:** To satisfy the `FinalDelivery` liveness property (Test T6), the server must acknowledge the highest contiguous sequence received, and the driver must retain frames until acknowledged.

10. Related work

This is an applied case study, not a new technique. TLA+ and its model checker are described in Lamport's *Specifying Systems*. Kleppmann's *Designing Data-Intensive Applications* covers ordering, idempotency, and replication trade-offs that this work applies to a concrete system. Lamport's foundational paper *"Time, Clocks, and the Ordering of Events in a Distributed System"* (1978) establishes the necessity of logical sequence numbers over physical clocks. Beschastnikh, Wang, Brun, and Ernst's *"Debugging distributed systems: Challenges and options for validation and debugging"* (Communications of the ACM, 2016) surveys the difficulty of validating distributed systems and tools for doing so.

Appendix A: Architecture and schema

Stack.

- Node.js with Express for ingestion.
- Socket.IO over secure WebSockets, scaled out with a Redis pub/sub adapter.
- PostgreSQL 15 with PostGIS.
- React Native apps for drivers and parents.

Schema.

- **Users** (user_id, email, password hash, role)
- **Routes** (route_id, name, start and end location)
- **Children** (child_id, parent, route, name)
- **Trips** (trip_id, driver, route, status, start time)

- **Locations** (loc_id, trip, coordinates, seq_id, recorded_at). Time-based partitioning by `recorded_at`.

Security.

- JWTs signed with RS256 and validated on both the API and the WebSocket handlers.
- Passwords hashed with bcrypt (cost factor at least 12).
- Role-based checks: drivers publish only to their assigned trip, parents listen only to their own child's trip.
- TLS 1.3 for HTTPS and secure WebSocket traffic.

Main design decisions.

- **WebSockets over polling.** Persistent connections avoid the repeated header overhead and latency of polling for hundreds of concurrent parents at pickup and drop-off times.
- **PostgreSQL with PostGIS over MongoDB.** Relational integrity (parent-child-route mappings) matters for safety, and spatial indexing is built in.
- **Shared ordering responsibility.** The driver app queues and numbers frames, the server filters stale ones, and the parent app guards its display. This keeps heavy reconciliation off low-power phones.